

LamzyTech Student News API

A Beginner's Guide to Connecting a Website to an API

Using plain HTML, CSS, and JavaScript with `fetch()` and `async/await`

Live API Base URL

<https://lamzytechnewsapi.onrender.com>

Prepared for IT students building their first API-powered project

What's Inside This Guide

- 1. Frontend vs. Backend — How Your Site Talks to the API
- 2. Two Concepts You Must Understand First: `fetch()` and `async/await`
- 3. Build One Reusable Helper Function (`api.js`)
- 4. Registering a New Student
- 5. Logging In and Out
- 6. Displaying Posts to Everyone (No Login Needed)
- 7. Creating a Post (Login Required)
- 8. Updating a Post (Author Only)
- 9. Deleting a Post (Author Only)
- 10. Viewing Your Own Profile
- 11. Handling Errors the Right Way
- 12. What Happens When Your Login Token Expires
- 13. Suggested Project Folder Structure
- 14. Where to Find a Full Working Example
- 15. Final Checklist Before You Submit

BEFORE YOU START

This guide assumes you already know basic HTML, CSS, and JavaScript. If "Promise", "JSON", or "async function" sound unfamiliar, read Section 2 slowly and try the small examples yourself before moving on.

1. Frontend vs. Backend: How Your Site Talks to the API

Your HTML/CSS/JS website is the "frontend" the part people see and click on in their browser. The LamzyTech News API is the "backend" a separate program running on a server that stores all the student accounts and news posts in a database.

These two things never touch each other directly. Your frontend cannot open the database itself. Instead, your JavaScript code sends a request over the internet to the API, and the API replies with data in JSON format. This request/response trip is called an HTTP request, and in JavaScript you make one using the built-in `fetch()` function.

The request flow

```
Your JS code → fetch(url, options) → API → JSON response → your JS updates the page
```

Every single response this API sends back — whether it succeeded or failed — follows the exact same shape:

```
{
  "success": true,
  "message": "Request successful",
  "data": { }
}
```

Because the shape never changes, your JavaScript can follow the same three steps after every fetch call, no matter which endpoint you're calling:

- Convert the raw response into JSON.
- Check whether success is true or false.
- If it's true, use whatever is inside data.

2. Two Concepts You Must Understand First

Before writing any API calls, make sure you're comfortable with these two JavaScript features. Almost everything else in this guide is built on top of them.

`fetch(url, options)`

Sends an HTTP request to a URL and returns a Promise — a placeholder object that represents a value which isn't ready yet, but will be at some point in the near future.

`async / await`

The `await` keyword pauses your function until a Promise finishes, and hands you back the actual result instead of chaining lots of confusing `.then()` calls. Any function that uses `await` must be declared with the `async` keyword.

Here is the minimum pattern you will repeat throughout this whole project:

```
async function example() {
  const response = await fetch("https://lamzytechnewsapi.onrender.com/api/posts");
  const result = await response.json(); // parse the JSON body
}
```

```
    console.log(result);  
  }
```

Notice the two awaits: one for the network trip itself (fetch), and a second one for reading the body of the response as JSON (response.json() is also asynchronous). Every function later in this guide follows this exact pattern.

3. Build One Reusable Helper Function (api.js)

Instead of retyping the same fetch boilerplate on every page, write ONE helper function that every other function calls. This gives you a single place to fix things later for example, if you ever change how the login token is stored.

Save the following as js/api.js:

```
// api.js
const BASE_URL = "https://lamzytechnewsapi.onrender.com";

/**
 * Generic helper for calling the LamzyTech News API.
 * @param {string} endpoint - e.g. "/api/posts"
 * @param {object} options - fetch options (method, body, etc.)
 * @param {boolean} authRequired - whether to attach the saved JWT
 */

async function apiRequest(endpoint, options = {}, authRequired = false) {
  const headers = {
    "Content-Type": "application/json",
    ...options.headers,
  };

  if (authRequired) {
    const token = localStorage.getItem("token");
    if (!token) {
      throw new Error("You must be logged in to do this.");
    }
    headers["Authorization"] = `Bearer ${token}`;
  }

  const response = await fetch(`${BASE_URL}${endpoint}`, {
    ...options,
    headers,
  });

  const result = await response.json();

  if (!result.success) {
    // The API always explains what went wrong in `message`
    throw new Error(result.message || "Something went wrong.");
  }

  return result.data;
}
```

4. Registering a New Student

Registration creates a student account and immediately logs them in the API returns a token (used to prove who they are on future requests) along with the new student's profile.

```
async function registerStudent(fullName, email, password, department, level) {
  const data = await apiRequest("/api/auth/register", {
    method: "POST",
    body: JSON.stringify({ fullName, email, password, department, level }),
  });
}
```

```

});

// data = { token, tokenType, student: {...} }
localStorage.setItem("token", data.token);
localStorage.setItem("student", JSON.stringify(data.student));

return data.student;
}

```

Example HTML form

```

<form id="registerForm">
  <input id="fullName" placeholder="Full name" required />
  <input id="email" type="email" placeholder="Email" required />
  <input id="password" type="password" placeholder="Password (min 6 chars)" required />
  <input id="department" placeholder="Department" required />
  <input id="level" placeholder="Level (e.g. 300)" required />
  <button type="submit">Register</button>
</form>

```

Wiring the form up to the function

```

document.getElementById("registerForm").addEventListener("submit", async (e) => {
  e.preventDefault(); // stop the page from reloading

  try {
    const student = await registerStudent(
      document.getElementById("fullName").value,
      document.getElementById("email").value,
      document.getElementById("password").value,
      document.getElementById("department").value,
      document.getElementById("level").value
    );
    alert(`Welcome, ${student.fullName}! You're now logged in.`);
    window.location.href = "index.html"; // go to the homepage
  } catch (err) {
    alert(err.message); // e.g. "A student with this email already exists"
  }
});

```

5. Logging In and Out

Logging in

Same idea as registration, but with a simpler payload:

```
async function loginStudent(email, password) {
  const data = await apiRequest("/api/auth/login", {
    method: "POST",
    body: JSON.stringify({ email, password }),
  });

  localStorage.setItem("token", data.token);
  localStorage.setItem("student", JSON.stringify(data.student));

  return data.student;
}

document.getElementById("loginForm").addEventListener("submit", async (e) => {
  e.preventDefault();
  try {
    await loginStudent(
      document.getElementById("email").value,
      document.getElementById("password").value
    );
    window.location.href = "index.html";
  } catch (err) {
    alert(err.message); // e.g. "Invalid email or password"
  }
});
```

Logging out

There is no /logout endpoint to call. Tokens (JWTs) are "stateless" — the server doesn't keep track of who is logged in. "Logging out" simply means deleting the token from the browser:

```
function logout() {
  localStorage.removeItem("token");
  localStorage.removeItem("student");
  window.location.href = "login.html";
}
```

Checking if someone is logged in

```
function getCurrentStudent() {
  const raw = localStorage.getItem("student");
  return raw ? JSON.parse(raw) : null;
}

function isLoggedIn() {
  return !!localStorage.getItem("token");
}
```

Use isLoggedIn() to decide whether your navigation bar should show "Login" or "Create Post" / "Logout".

6. Displaying Posts to Everyone (No Login Needed)

Get all posts (paginated)

```
async function getAllPosts(page = 0, size = 10) {
  return await apiRequest(`/api/posts?page=${page}&size=${size}`);
  // returns a Spring "Page" object: { content: [...], totalPages, totalElements, number, ... }
}
```

Rendering posts onto the page

```
<div id="postsList"></div>

async function renderPosts() {
  const container = document.getElementById("postsList");
  container.innerHTML = "Loading posts...";

  try {
    const page = await getAllPosts(0, 10);

    if (page.content.length === 0) {
      container.innerHTML = "<p>No posts yet.</p>";
      return;
    }

    container.innerHTML = page.content
      .map(
        (post) => `
<article class="post-card">
  ${post.imageUrl ? `` :
    ""}
    <span class="category">${escapeHtml(post.category)}</span>
    <h2><a
href="post.html?id=${post.id}">${escapeHtml(post.title)}</a></h2>
    <p>${escapeHtml(post.content.slice(0, 150))}...</p>
    <small>By ${escapeHtml(post.author.fullName)} · ${new
Date(post.createdAt).toLocaleDateString()}</small>
    </article>
  `
      )
      .join("");
  } catch (err) {
    container.innerHTML = `<p>Failed to load posts: ${err.message}</p>`;
  }
}

renderPosts();

// Always escape user-supplied text before inserting it as HTML,
// so a post title/content can never inject a <script> tag into your page.
function escapeHtml(str) {
  const div = document.createElement("div");
  div.textContent = str;
  return div.innerHTML;
}
```

Get a single post (e.g. post.html?id=5)

```
async function getPostById(id) {
  return await apiRequest(`/api/posts/${id}`);
}

async function renderSinglePost() {
  const params = new URLSearchParams(window.location.search);
  const id = params.get("id");
  const container = document.getElementById("postDetail");

  try {
    const post = await getPostById(id);
    container.innerHTML = `
      <h1>${escapeHtml(post.title)}</h1>
      <small>By ${escapeHtml(post.author.fullName)} – ${escapeHtml(post.category)}</small>
      ${post.imageUrl ? `` : ""}
      <p>${escapeHtml(post.content)}</p>
    `;
  } catch (err) {
    container.innerHTML = `<p>Post not found.</p>`;
  }
}

renderSinglePost();
```

7. Creating a Post (Login Required)

```
async function createPost(title, content, category, imageUrl) {
  return await apiRequest(
    "/api/posts",
    {
      method: "POST",
      body: JSON.stringify({ title, content, category, imageUrl }),
    },
    true // authRequired = true, attaches the JWT
  );
}

document.getElementById("createPostForm").addEventListener("submit", async (e) => {
  e.preventDefault();

  if (!isLoggedIn()) {
    alert("You need to log in first.");
    window.location.href = "login.html";
    return;
  }

  try {
    const post = await createPost(
      document.getElementById("title").value,
      document.getElementById("content").value,
      document.getElementById("category").value,
      document.getElementById("imageUrl").value // can be empty
    );
    window.location.href = `post.html?id=${post.id}`;
  } catch (err) {
    alert(err.message); // e.g. "Title is required"
  }
});
```

8. Updating a Post (Author Only)

```
async function updatePost(id, title, content, category, imageUrl) {
  return await apiRequest(
    `/api/posts/${id}`,
    {
      method: "PUT",
      body: JSON.stringify({ title, content, category, imageUrl }),
    },
    true
  );
}
```

If a logged-in student who is NOT the original author tries to call this, the API responds with 403 Forbidden, and `apiRequest()` will throw an error with the API's own message. Always catch it and show it to the user instead of letting the page silently fail.

9. Deleting a Post (Author Only)

```
async function deletePost(id) {
  return await apiRequest(`/api/posts/${id}`, { method: "DELETE" }, true);
}

document.getElementById("deleteBtn").addEventListener("click", async () => {
  if (!confirm("Delete this post? This can't be undone.")) return;

  try {
    await deletePost(currentPostId);
    alert("Post deleted.");
    window.location.href = "index.html";
  } catch (err) {
    alert(err.message);
  }
});
```

Only showing Edit/Delete buttons to the author

```
function renderPostActions(post) {
  const student = getCurrentStudent();
  const isAuthor = student && student.id === post.author.id;

  return isAuthor
    ? `
```

10. Viewing Your Own Profile

```
async function getMyProfile() {
  return await apiRequest("/api/students/me", {}, true);
}
```

This is handy for a "My Account" page, or simply to confirm that a saved token is still valid before showing a form.

11. Handling Errors the Right Way

Because `apiRequest()` throws a normal JavaScript Error containing the API's own message whenever success is false, your UI code only ever needs one pattern, everywhere in your project:

```
try {
  const data = await someApiCall();
  // success path
} catch (err) {
  showErrorToUser(err.message);
}
```

Common messages you will see

err.message (roughly)	What happened	Status
Field-specific, e.g. "Title is required"	Validation failed	400
"Invalid email or password"	Wrong login credentials	401
"You must be logged in to do this."	Thrown before the API is even called no token in localStorage	—
"Unauthorized: authentication is required..."	Token missing or expired on a protected route	401
"You are not allowed to modify a post you did not author"	Editing/deleting someone else's post	403
"News post not found with id: X"	Bad or deleted post ID	404
"A student with this email already exists"	Duplicate registration	409

12. What Happens When Your Login Token Expires

Tokens expire 24 hours after login. If a student's session goes stale while they're still on the site, any authenticated call will fail with a 401 status. Handle this gracefully in one place — inside `apiRequest()` itself:

```
async function apiRequest(endpoint, options = {}, authRequired = false) {
  // ...same as before...

  const response = await fetch(`${BASE_URL}${endpoint}`, { ...options, headers });
  const result = await response.json();

  if (!result.success) {
    if (response.status === 401 && authRequired) {
      // Token is bad/expired – clear it and send them to log in again
      localStorage.removeItem("token");
      localStorage.removeItem("student");
      alert("Your session has expired. Please log in again.");
      window.location.href = "login.html";
      return;
    }
    throw new Error(result.message || "Something went wrong.");
  }

  return result.data;
}
```

13. Suggested Project Folder Structure

```
news-website/
├── index.html          (list of posts – homepage)
├── post.html           (single post view)
├── login.html
├── register.html
├── create-post.html
├── edit.html
├── css/
│   └── style.css
├── js/
│   ├── api.js          (the apiRequest() helper + all endpoint functions)
│   ├── home.js         (renderPosts(), used on index.html)
│   ├── post.js         (renderSinglePost(), used on post.html)
│   ├── auth.js         (register/login form handlers)
│   └── post-form.js    (create/edit form handlers)
```

Load `api.js` BEFORE your page-specific script in every HTML file:

```
<script src="js/api.js"></script>
<script src="js/home.js"></script>
```

14. Where to Find a Full Working Example

A minimal but complete example: homepage, login, and create-post; already wired up to the live API is included alongside this guide in the demo-website/ folder. Open demo-website/index.html directly in your browser (no server needed it's plain HTML/CSS/JS) to see it working end-to-end, then use it as a reference while you build your own version.

15. Final Checklist Before You Submit

- ☐ api.js has one apiRequest() helper, no repeated fetch boilerplate
- ☐ Token is saved to localStorage on register/login, and removed on logout
- ☐ Every user-generated string (post.title, post.content, etc.) is escaped before being inserted into the page — never build HTML with raw template-literal interpolation of API data
- ☐ Every fetch call is wrapped in try/catch, and errors are shown to the user, not just logged to the console
- ☐ Edit/Delete buttons only render for the post's author, but a 403 is still handled gracefully in case someone bypasses that in dev tools
- ☐ A loading state is shown while waiting on the first request (cold start can take up to a minute)